Build Task — Complete SQL Solutions & QA Test Cases

Part 1: 89 Solved SQL Queries (SALESPEOPLE, CUST, ORDERS) | Part 2: Magnus Login Types Test Suite

Part 1: SQL Practice Queries — Complete Solutions (89 Queries)

Database Schema: SALESPEOPLE (SNUM, SNAME, CITY, COMM) • **CUST** (CNUM, CNAME, CITY, RATING, SNUM) • **ORDERS** (ONUM, AMT, ODATE, CNUM, SNUM)

Q01 **Display snum, sname, city and comm of all salespeople.**

```
SELECT snum, sname, city, comm
FROM SALESPEOPLE;
```

Q02 **Display all snum without duplicates from all orders.**

```
SELECT DISTINCT snum
FROM ORDERS;
```

Q03 **Display names and commissions of all salespeople in London.**

```
SELECT sname, comm
FROM SALESPEOPLE
WHERE city = 'London';
```

Q04 **All customers with rating of 100.**

```
SELECT *
FROM CUST
WHERE rating = 100;
```

Q05 **Produce orderno, amount and date from all rows in the order table.**

```
SELECT onum, amt, odate
FROM ORDERS;
```

Q06 **All customers in San Jose, who have rating more than 200.**

```
SELECT *
FROM CUST
WHERE city = 'San Jose' AND rating > 200;
```

Q07 **All customers who were either located in San Jose or had a rating above 200.**

```
SELECT *
FROM CUST
WHERE city = 'San Jose' OR rating > 200;
```

Q08 **All orders for more than $1000.**

```
SELECT *
FROM ORDERS
WHERE amt > 1000;
```

Q09 **Names and cities of all salespeople in London with commission above 0.10.**

```
SELECT sname, city
FROM SALESPEOPLE
WHERE city = 'London' AND comm > 0.10;
```

Q10 **All customers excluding those with rating <= 100 unless they are located in Rome.**

```
SELECT *
FROM CUST
WHERE rating > 100 OR city = 'Rome';
```

Q11 **All salespeople either in Barcelona or in London.**

```
SELECT *
FROM SALESPEOPLE
WHERE city IN ('Barcelona', 'London');
```

Q12 **All salespeople with commission between 0.10 and 0.12. (Boundary values should be excluded)**

```
SELECT *
FROM SALESPEOPLE
WHERE comm > 0.10 AND comm < 0.12;
```

Q13 **All customers with NULL values in city column.**

```
SELECT *
FROM CUST
WHERE city IS NULL;
```

Q14 **All orders taken on Oct 3rd and Oct 4th 1994.**

```
SELECT *
FROM ORDERS
WHERE odate IN ('03-OCT-94', '04-OCT-94');
```

Q15 **All customers serviced by Peel or Motika.**

```
SELECT c.*
FROM CUST c
JOIN SALESPEOPLE s ON c.snum = s.snum
WHERE s.sname IN ('Peel', 'Motika');
```

Q16 **All customers whose names begin with a letter from A to B.**

```
SELECT *
FROM CUST
WHERE cname >= 'A' AND cname < 'C';
-- Alternative: WHERE cname LIKE 'A%' OR cname LIKE 'B%';
```

Q17 **All orders except those with 0 or NULL value in amt field.**

```
SELECT *
FROM ORDERS
WHERE amt IS NOT NULL AND amt <> 0;
```

Q18 **Count the number of salespeople currently listing orders in the order table.**

```
SELECT COUNT(DISTINCT snum) AS active_salespeople
FROM ORDERS;
```

Q19 **Largest order taken by each salesperson, datewise.**

```
SELECT snum, odate, MAX(amt) AS largest_order
FROM ORDERS
GROUP BY snum, odate;
```

Q20 **Largest order taken by each salesperson with order value more than $3000.**

```
SELECT snum, MAX(amt) AS largest_order
FROM ORDERS
GROUP BY snum
HAVING MAX(amt) > 3000;
```

Q21 **Which day had the highest total amount ordered.**

```
SELECT odate, SUM(amt) AS total_amt
FROM ORDERS
GROUP BY odate
HAVING SUM(amt) = (
    SELECT MAX(daily_total)
    FROM (
        SELECT SUM(amt) AS daily_total
        FROM ORDERS
        GROUP BY odate
    ) t
);
```

Q22 **Count all orders for Oct 3rd.**

```
SELECT COUNT(*) AS oct3_orders
FROM ORDERS
WHERE odate = '03-OCT-94';
```

Q23 **Count the number of different non NULL city values in customers table.**

```
SELECT COUNT(DISTINCT city) AS distinct_cities
FROM CUST;
```

Q24 **Select each customer’s smallest order.**

```
SELECT cnum, MIN(amt) AS smallest_order
FROM ORDERS
GROUP BY cnum;
```

Q25 **First customer in alphabetical order whose name begins with G.**

```
SELECT MIN(cname) AS first_g_customer
FROM CUST
WHERE cname LIKE 'G%';
```

Q26 **Get the output like “For dd/mm/yy there are ___ orders.”**

```
SELECT 'For ' || odate || ' there are ' || COUNT(*) || ' orders.' AS order_summary
FROM ORDERS
GROUP BY odate;
-- MySQL: SELECT CONCAT('For ', odate, ' there are ', COUNT(*), ' orders.') FROM ORDERS GROUP BY odate;
```

Q27 **Assume that each salesperson has a 12% commission. Produce order no., salesperson no., and amount of salesperson’s commission for that order.**

```
SELECT onum, snum, ROUND(amt * 0.12, 2) AS commission_amt
FROM ORDERS;
```

Q28 **Find highest rating in each city. Put the output in this form: For the city (city), the highest rating is : (rating).**

```
SELECT 'For the city ' || city || ', the highest rating is : ' || MAX(rating) AS result
FROM CUST
GROUP BY city;
-- MySQL: SELECT CONCAT('For the city ', city, ', the highest rating is : ', MAX(rating)) FROM CUST
GROUP BY city;
```

Q29 **Display the totals of orders for each day and place the results in descending order.**

```
SELECT odate, SUM(amt) AS total_amount
FROM ORDERS
GROUP BY odate
ORDER BY total_amount DESC;
```

Q30 **All combinations of salespeople and customers who shared a city. (i.e. same city).**

```
SELECT s.sname, c.cname, s.city
FROM SALESPEOPLE s
JOIN CUST c ON s.city = c.city;
```

Q31 **Name of all customers matched with the salespeople serving them.**

```
SELECT c.cname, s.sname
FROM CUST c
JOIN SALESPEOPLE s ON c.snum = s.snum;
```

Q32 **List each order number followed by the name of the customer who made the order.**

```
SELECT o.onum, c.cname
FROM ORDERS o
JOIN CUST c ON o.cnum = c.cnum;
```

Q33 **Names of salesperson and customer for each order after the order number.**

```
SELECT o.onum, s.sname, c.cname
FROM ORDERS o
JOIN SALESPEOPLE s ON o.snum = s.snum
JOIN CUST c ON o.cnum = c.cnum;
```

Q34 **Produce all customers serviced by salespeople with a commission above 12%.**

```
SELECT c.*
FROM CUST c
JOIN SALESPEOPLE s ON c.snum = s.snum
WHERE s.comm > 0.12;
```

Q35 **Calculate the amount of the salesperson’s commission on each order with a rating above 100.**

```
SELECT o.onum, s.sname, ROUND(o.amt * s.comm, 2) AS commission_amt
FROM ORDERS o
JOIN CUST c ON o.cnum = c.cnum
JOIN SALESPEOPLE s ON o.snum = s.snum
WHERE c.rating > 100;
```

Q36 **Find all pairs of customers having the same rating.**

```
SELECT a.cname AS customer_1, b.cname AS customer_2, a.rating
FROM CUST a
JOIN CUST b ON a.rating = b.rating
WHERE a.cnum <> b.cnum;
```

Q37 **Find all pairs of customers having the same rating, each pair coming once only.**

```
SELECT a.cname AS customer_1, b.cname AS customer_2, a.rating
FROM CUST a
JOIN CUST b ON a.rating = b.rating
WHERE a.cnum < b.cnum;
```

Q38 **Policy is to assign three salespeople to each customer. Display all such combinations.**

```
SELECT c.cname, s1.sname AS sp1, s2.sname AS sp2, s3.sname AS sp3
FROM CUST c
CROSS JOIN SALESPEOPLE s1
JOIN SALESPEOPLE s2 ON s1.snum < s2.snum
JOIN SALESPEOPLE s3 ON s2.snum < s3.snum;
```

Q39 **Display all customers located in cities where salesman Serres has a customer.**

```
SELECT *
FROM CUST
WHERE city IN (
    SELECT c.city
    FROM CUST c
    JOIN SALESPEOPLE s ON c.snum = s.snum
    WHERE s.sname = 'Serres'
);
```

Q40 **Find all pairs of customers served by a single salesperson.**

```
SELECT a.cname AS customer_1, b.cname AS customer_2, a.snum
FROM CUST a
JOIN CUST b ON a.snum = b.snum
WHERE a.cnum < b.cnum;
```

Q41 **Produce all pairs of salespeople who are living in the same city. Exclude combinations of salespeople with themselves as well as duplicates with the order reversed.**

```
SELECT a.sname AS salesperson_1, b.sname AS salesperson_2, a.city
FROM SALESPEOPLE a
JOIN SALESPEOPLE b ON a.city = b.city
WHERE a.snum < b.snum;
```

Q42 **Produce all pairs of orders by a given customer, name that customer and eliminate duplicates.**

```
SELECT c.cname, a.onum AS order_1, b.onum AS order_2
FROM ORDERS a
JOIN ORDERS b ON a.cnum = b.cnum AND a.onum < b.onum
JOIN CUST c ON a.cnum = c.cnum;
```

Q43 **Produce names and cities of all customers with the same rating as Hoffman.**

```
SELECT cname, city
FROM CUST
WHERE rating = (
    SELECT rating
    FROM CUST
    WHERE cname = 'Hoffman'
);
```

Q44 **Extract all the orders of Motika.**

```
SELECT o.*
FROM ORDERS o
JOIN SALESPEOPLE s ON o.snum = s.snum
WHERE s.sname = 'Motika';
```

Q45 **All orders credited to the same salesperson who services Hoffman.**

```
SELECT *
FROM ORDERS
WHERE snum = (
    SELECT snum
    FROM CUST
    WHERE cname = 'Hoffman'
);
```

Q46 **All orders that are greater than the average for Oct 4.**

```
SELECT *
FROM ORDERS
WHERE amt > (
    SELECT AVG(amt)
    FROM ORDERS
    WHERE odate = '04-OCT-94'
);
```

Q47 **Find average commission of salespeople in London.**

```
SELECT AVG(comm) AS avg_london_comm
FROM SALESPEOPLE
WHERE city = 'London';
```

Q48 **Find all orders attributed to salespeople servicing customers in London.**

```
SELECT *
FROM ORDERS
WHERE snum IN (
    SELECT DISTINCT snum
    FROM CUST
    WHERE city = 'London'
);
```

Q49 **Extract commissions of all salespeople servicing customers in London.**

```
SELECT DISTINCT s.snum, s.sname, s.comm
FROM SALESPEOPLE s
JOIN CUST c ON s.snum = c.snum
WHERE c.city = 'London';
```

Q50 **Find all customers whose cnum is 1000 above the snum of Serres.**

```
SELECT *
FROM CUST
WHERE cnum = (
    SELECT snum + 1000
    FROM SALESPEOPLE
    WHERE sname = 'Serres'
);
```

Q51 **Count the customers with rating above San Jose's average.**

```
SELECT COUNT(*) AS above_sj_avg
FROM CUST
WHERE rating > (
    SELECT AVG(rating)
    FROM CUST
    WHERE city = 'San Jose'
);
```

Q52 **Obtain all orders for the customer named Cisnerous. (Assume you don't know his customer no. (cnum)).**

```
SELECT o.*
FROM ORDERS o
JOIN CUST c ON o.cnum = c.cnum
WHERE c.cname = 'Cisnerous';
```

Q53 **Produce the names and rating of all customers who have above average orders.**

```
SELECT DISTINCT c.cname, c.rating
FROM CUST c
JOIN ORDERS o ON c.cnum = o.cnum
WHERE o.amt > (SELECT AVG(amt) FROM ORDERS);
```

Q54 **Find total amount in orders for each salesperson for whom this total is greater than the amount of the largest order in the table.**

```
SELECT snum, SUM(amt) AS total_amt
FROM ORDERS
GROUP BY snum
HAVING SUM(amt) > (SELECT MAX(amt) FROM ORDERS);
```

Q55 **Find all customers with an order on 3rd Oct.**

```
SELECT DISTINCT c.*
FROM CUST c
JOIN ORDERS o ON c.cnum = o.cnum
WHERE o.odate = '03-OCT-94';
```

Q56 **Find names and numbers of all salespeople who have more than one customer.**

```
SELECT s.snum, s.sname
FROM SALESPEOPLE s
JOIN CUST c ON s.snum = c.snum
GROUP BY s.snum, s.sname
HAVING COUNT(c.cnum) > 1;
```

Q57 **Check if the correct salesperson was credited with each sale.**

```
SELECT o.onum, o.cnum, o.snum AS order_snum, c.snum AS assigned_snum,
       CASE WHEN o.snum = c.snum THEN 'Correct' ELSE 'Mismatch' END AS credit_status
FROM ORDERS o
JOIN CUST c ON o.cnum = c.cnum;
```

Q58 **Find all orders with above average amounts for their customers.**

```
SELECT *
FROM ORDERS o1
WHERE amt > (
    SELECT AVG(amt)
    FROM ORDERS o2
    WHERE o2.cnum = o1.cnum
);
```

Q59 **Find the sums of the amounts from order table grouped by date, eliminating all those dates where the sum was not at least 2000 above the maximum amount.**

```
SELECT odate, SUM(amt) AS total_amt
FROM ORDERS
GROUP BY odate
HAVING SUM(amt) >= MAX(amt) + 2000;
```

Q60 **Find names and numbers of all customers with ratings equal to the maximum for their city.**

```
SELECT cnum, cname
FROM CUST c1
WHERE rating = (
    SELECT MAX(rating)
    FROM CUST c2
    WHERE c2.city = c1.city
);
```

Q61 **Find all salespeople who have customers in their cities who they don’t service. (Both ways: using Join and Correlated subquery.)**

```
-- Method 1: Using JOIN
SELECT DISTINCT s.snum, s.sname, s.city
FROM SALESPEOPLE s
JOIN CUST c ON s.city = c.city
WHERE s.snum <> c.snum;

-- Method 2: Using Correlated Subquery
SELECT snum, sname, city
FROM SALESPEOPLE s
WHERE EXISTS (
    SELECT 1
    FROM CUST c
    WHERE c.city = s.city AND c.snum <> s.snum
);
```

Q62 **Extract cnum, cname and city from customer table if and only if one or more of the customers in the table are located in San Jose.**

```
SELECT cnum, cname, city
FROM CUST
WHERE EXISTS (
    SELECT 1
    FROM CUST
    WHERE city = 'San Jose'
);
```

Q63 **Find salespeople no. who have multiple customers.**

```
SELECT snum
FROM CUST
GROUP BY snum
HAVING COUNT(cnum) > 1;
```

Q64 **Find salespeople number, name and city who have multiple customers.**

```
SELECT s.snum, s.sname, s.city
FROM SALESPEOPLE s
JOIN CUST c ON s.snum = c.snum
GROUP BY s.snum, s.sname, s.city
HAVING COUNT(c.cnum) > 1;
```

Q65 **Find salespeople who serve only one customer.**

```
SELECT s.snum, s.sname, s.city
FROM SALESPEOPLE s
JOIN CUST c ON s.snum = c.snum
GROUP BY s.snum, s.sname, s.city
HAVING COUNT(c.cnum) = 1;
```

Q66 **Extract rows of all salespeople with more than one current order.**

```
SELECT s.*
FROM SALESPEOPLE s
JOIN ORDERS o ON s.snum = o.snum
GROUP BY s.snum, s.sname, s.city, s.comm
HAVING COUNT(o.onum) > 1;
```

Q67 **Find all salespeople who have customers with a rating of 300. (use EXISTS)**

```
SELECT *
FROM SALESPEOPLE s
WHERE EXISTS (
    SELECT 1
    FROM CUST c
    WHERE c.snum = s.snum AND c.rating = 300
);
```

Q68 **Find all salespeople who have customers with a rating of 300. (use Join)**

```
SELECT DISTINCT s.*
FROM SALESPEOPLE s
JOIN CUST c ON s.snum = c.snum
WHERE c.rating = 300;
```

Q69 **Select all salespeople with customers located in their cities who are not assigned to them. (use EXISTS)**

```
SELECT *
FROM SALESPEOPLE s
WHERE EXISTS (
    SELECT 1
    FROM CUST c
    WHERE c.city = s.city AND c.snum <> s.snum
);
```

Q70 **Extract from customers table every customer assigned to a salesperson who currently has at least one other customer (besides the customer being selected) with orders in order table.**

```
SELECT *
FROM CUST c1
WHERE EXISTS (
    SELECT 1
    FROM CUST c2
    JOIN ORDERS o ON c2.cnum = o.cnum
    WHERE c2.snum = c1.snum AND c2.cnum <> c1.cnum
);
```

Q71 **Find salespeople with customers located in their cities (using both ANY and IN).**

```
-- Using ANY
SELECT *
FROM SALESPEOPLE
WHERE city = ANY (SELECT city FROM CUST);

-- Using IN
SELECT *
FROM SALESPEOPLE
WHERE city IN (SELECT city FROM CUST);
```

Q72 **Find all salespeople for whom there are customers that follow them in alphabetical order. (Using ANY and EXISTS)**

```
-- Using ANY
SELECT *
FROM SALESPEOPLE
WHERE sname < ANY (SELECT cname FROM CUST);

-- Using EXISTS
SELECT *
FROM SALESPEOPLE s
WHERE EXISTS (
    SELECT 1
    FROM CUST c
    WHERE c.cname > s.sname
);
```

Q73 **Select customers who have a greater rating than any customer in Rome.**

```
SELECT *
FROM CUST
WHERE rating > ANY (
    SELECT rating
    FROM CUST
    WHERE city = 'Rome'
);
```

Q74 **Select all orders that had amounts that were greater than at least one of the orders from Oct 6th.**

```
SELECT *
FROM ORDERS
WHERE amt > ANY (
    SELECT amt
    FROM ORDERS
    WHERE odate = '06-OCT-94'
);
```

Q75 **Find all orders with amounts smaller than any amount for a customer in San Jose. (Both using ANY and without ANY)**

```
-- Using ANY
SELECT *
FROM ORDERS
WHERE amt < ANY (
    SELECT o.amt
    FROM ORDERS o
    JOIN CUST c ON o.cnum = c.cnum
    WHERE c.city = 'San Jose'
);

-- Without ANY
SELECT *
FROM ORDERS
WHERE amt < (
    SELECT MAX(o.amt)
    FROM ORDERS o
    JOIN CUST c ON o.cnum = c.cnum
    WHERE c.city = 'San Jose'
);
```

Q76 **Select those customers whose ratings are higher than every customer in Paris. (Using both ALL and NOT EXISTS)**

```
-- Using ALL
SELECT *
FROM CUST
WHERE rating > ALL (
    SELECT rating
    FROM CUST
    WHERE city = 'Paris'
);

-- Using NOT EXISTS
SELECT *
FROM CUST c1
WHERE NOT EXISTS (
    SELECT 1
    FROM CUST c2
    WHERE c2.city = 'Paris' AND c2.rating >= c1.rating
);
```

Q77 **Select all customers whose ratings are equal to or greater than ANY of the Serres.**

```
SELECT *
FROM CUST
WHERE rating >= ANY (
    SELECT c.rating
    FROM CUST c
    JOIN SALESPEOPLE s ON c.snum = s.snum
    WHERE s.sname = 'Serres'
);
```

Q78 **Find all salespeople who have no customers located in their city. (Both using ANY and ALL)**

```
-- Using ANY
SELECT *
FROM SALESPEOPLE
WHERE NOT (city = ANY (SELECT city FROM CUST WHERE city IS NOT NULL));

-- Using ALL
SELECT *
FROM SALESPEOPLE
WHERE city <> ALL (SELECT city FROM CUST WHERE city IS NOT NULL);
```

Q79 **Find all orders for amounts greater than any for the customers in London.**

```
SELECT *
FROM ORDERS
WHERE amt > ANY (
    SELECT o.amt
    FROM ORDERS o
    JOIN CUST c ON o.cnum = c.cnum
    WHERE c.city = 'London'
);
```

Q80 **Find all salespeople and customers located in London.**

```
SELECT snum AS id, sname AS name, 'Salesperson' AS role
FROM SALESPEOPLE
WHERE city = 'London'
UNION
SELECT cnum AS id, cname AS name, 'Customer' AS role
FROM CUST
WHERE city = 'London';
```

Q81 **For every salesperson, dates on which highest and lowest orders were brought.**

```
SELECT o1.snum, o1.odate, o1.amt, 'Highest' AS order_type
FROM ORDERS o1
WHERE o1.amt = (SELECT MAX(amt) FROM ORDERS o2 WHERE o2.snum = o1.snum)
UNION
SELECT o1.snum, o1.odate, o1.amt, 'Lowest' AS order_type
FROM ORDERS o1
WHERE o1.amt = (SELECT MIN(amt) FROM ORDERS o2 WHERE o2.snum = o1.snum)
ORDER BY snum, order_type;
```

Q82 **List all of the salespeople and indicate those who don’t have customers in their cities as well as those who do have.**

```
SELECT DISTINCT s.snum, s.sname, s.city, 'Matched Customer in City' AS city_status
FROM SALESPEOPLE s
JOIN CUST c ON s.city = c.city
UNION
SELECT s.snum, s.sname, s.city, 'No Customer in City' AS city_status
FROM SALESPEOPLE s
WHERE NOT EXISTS (SELECT 1 FROM CUST c WHERE c.city = s.city);
```

Q83 **Append strings to the selected fields, indicating whether or not a given salesperson was matched to a customer in his city.**

```
SELECT s.snum, s.sname, s.city || ' - Matched' AS status
FROM SALESPEOPLE s
WHERE EXISTS (SELECT 1 FROM CUST c WHERE c.city = s.city)
UNION
SELECT s.snum, s.sname, s.city || ' - Not Matched' AS status
FROM SALESPEOPLE s
WHERE NOT EXISTS (SELECT 1 FROM CUST c WHERE c.city = s.city);
```

Q84 **Create a union of two queries that shows the names, cities and ratings of all customers. Those with a rating of 200 or greater will also have the words ‘High Rating’, while the others will have the words ‘Low Rating’.**

```
SELECT cname, city, rating, 'High Rating' AS rating_category
FROM CUST
WHERE rating >= 200
UNION
SELECT cname, city, rating, 'Low Rating' AS rating_category
FROM CUST
WHERE rating < 200 OR rating IS NULL;
```

Q85 **Write command that produces the name and number of each salesperson and each customer with more than one current order. Put the result in alphabetical order.**

```
SELECT s.snum AS number, s.sname AS name
FROM SALESPEOPLE s
JOIN ORDERS o ON s.snum = o.snum
GROUP BY s.snum, s.sname
HAVING COUNT(o.onum) > 1
UNION
SELECT c.cnum AS number, c.cname AS name
FROM CUST c
JOIN ORDERS o ON c.cnum = o.cnum
GROUP BY c.cnum, c.cname
HAVING COUNT(o.onum) > 1
ORDER BY name ASC;
```

Q86 **Form a union of three queries. Have the first select the snums of all salespeople in San Jose, the second the cnums of all customers in San Jose and the third the onums of all orders on Oct. 3. Retain duplicates between the last two queries, but eliminate any redundancies between either of them and the first.**

```
SELECT snum AS id FROM SALESPEOPLE WHERE city = 'San Jose'
UNION
(
    SELECT cnum AS id FROM CUST WHERE city = 'San Jose'
    UNION ALL
    SELECT onum AS id FROM ORDERS WHERE odate = '03-OCT-94'
);
```

Q87 **Produce all the salespeople in London who had at least one customer there.**

```
SELECT DISTINCT s.*
FROM SALESPEOPLE s
JOIN CUST c ON s.city = c.city
WHERE s.city = 'London';
```

Q88 **Produce all the salespeople in London who did not have customers there.**

```
SELECT *
FROM SALESPEOPLE s
WHERE city = 'London'
  AND NOT EXISTS (
      SELECT 1
      FROM CUST c
      WHERE c.city = s.city
  );
```

Q89 **We want to see salespeople matched to their customers without excluding those salespeople who were not currently assigned to any customers. (Use OUTER join and UNION)**

```
-- Method 1: Using LEFT OUTER JOIN
SELECT s.snum, s.sname, c.cnum, c.cname
FROM SALESPEOPLE s
LEFT OUTER JOIN CUST c ON s.snum = c.snum;

-- Method 2: Using UNION
SELECT s.snum, s.sname, c.cnum, c.cname
FROM SALESPEOPLE s
JOIN CUST c ON s.snum = c.snum
UNION
SELECT s.snum, s.sname, NULL AS cnum, NULL AS cname
FROM SALESPEOPLE s
WHERE NOT EXISTS (SELECT 1 FROM CUST c WHERE c.snum = s.snum);
```

Part 2: QA Test Case Build — Magnus (Login Types)

Project / Component	Details	Reference & Sign-Off	Version & Review Status
Application Name	Magnus	**FSD Document**	Magnus_FSD_2.0 (v2.0 — Signed-off: Yes)
Component / Feature	Login Types	**Test Scenario Doc**	Magnus_TestScenarios_2.2 (v2.2 — Signed-off: Yes)
Primary Owner	Bharath	**Peer Review**	Neeraja (Reviewed: 2017-04-05 / 2017-04-08)
Secondary Owner	Neeraja	**BA Review & Version**	John (Reviewed: 2017-04-14) | Current Build: v1.2

#	Scenario / TC ID	Flags	Description & Pre-Conditions	Execution Steps	Expected Result	Status	Owner
1	**TC_LoginTypes_01** TS_LoginTypes_01	Smoke: Yes UAT: Yes	**Verify that the Admin user is able to Create an employee** Pre: Admin credentials available; Application URL accessible.	1. Click Admin link on the top right corner and Login with Admin user. 2. Navigate to Employee > Create and enter all mandatory fields. 3. Click Submit button and verify database entry.	1. User logs in successfully. 2. Employee registration succeeds. 3. Registered user is added to database.	**Pass** Working as expected	Bharath -
2	**TC_LoginTypes_02** TS_LoginTypes_01	Smoke: No UAT: No	**Verify that the Admin user is able to perform Employee search** Pre: Admin logged in; Employee records exist in database.	1. Click Employee Search button from menu. 2. Search by partial/ complete phone number and partial/complete email. 3. Search with combined criteria and click Clear button.	1. Employee Search page opens. 2. Matching records display for phone/ email searches. 3. Clear button resets all search fields.	**Fail** Server error on clicking Search	Bharath Mag-21

3	**TC_LoginTypes_03** TS_LoginTypes_01	Smoke: Yes UAT: Yes	**Verify Admin Login with valid username and valid password** Pre: User is on Magnus Login page.	1. Navigate to Magnus Login URL. 2. Enter valid Admin Email/Username and valid Password. 3. Click Sign In button.	User is authenticated and redirected to the Admin Home Dashboard.	**Pass** Working as expected	Bharath -
4	**TC_LoginTypes_04** TS_LoginTypes_01	Smoke: Yes UAT: No	**Verify Admin Login validation with invalid password or unregistered email** Pre: User is on Magnus Login page.	1. Enter valid Admin email and incorrect password; click Sign In. 2. Enter unregistered email and any password; click Sign In.	System denies access and displays 'Invalid Username or Password' error alert.	**Pass** Working as expected	Bharath -
5	**TC_LoginTypes_05** TS_LoginTypes_01	Smoke: No UAT: No	**Verify Login behavior with blank/empty mandatory credentials** Pre: User is on Magnus Login page.	1. Leave Email and Password fields empty. 2. Click Sign In button.	Inline validation message prompts user to enter Email and Password; form does not submit.	**Pass** Working as expected	Neeraja -
6	**TC_LoginTypes_06** TS_LoginTypes_02	Smoke: Yes UAT: Yes	**Verify Forgot Password link sends password recovery instructions** Pre: Registered user email exists; user is on Login page.	1. Click 'Forgot Password' link on Login screen. 2. Enter registered Email ID and click Submit.	Confirmation message displayed and password reset email triggered.	**Pass** Working as expected	Neeraja -
7	**TC_LoginTypes_07** TS_LoginTypes_02	Smoke: No UAT: Yes	**Verify Remember Me checkbox persists session credentials across browser restart** Pre: User is on Magnus Login page.	1. Enter valid credentials and check 'Remember Me'. 2. Sign in, close browser tab, and reopen application URL.	User session remains active or username is pre-populated on login screen.	**Pass** Working as expected	Bharath -
8	**TC_LoginTypes_08** TS_LoginTypes_02	Smoke: Yes UAT: Yes	**Verify Logout functionality terminates active user session securely** Pre: Admin user is logged into Magnus application.	1. Click Logout link in the top navigation header. 2. Click browser Back button after logout.	User is redirected to Login page; Back button does not restore authenticated pages.	**Pass** Working as expected	Bharath -